verifica la correttezza ed applicablita' della soluzione proposta nella CR change-request-docker-compose-docker-dev.md rispetto alla versione attuale del comando realizzata con i files allegati.

change-request-docker-compose-docker-dev.md
docker-dev/docker-dev.sh
docker-dev/apply-templates.sh
docker-dev/docker-run-extra-args.sh
docker-dev/langs/Python/templates/Dockerfile
docker-dev/langs/Python/templates/docker-run-extra-args
docker-dev/langs/Python/templates/entrypoint.Python.sh
docker-dev/langs/Python/bin/10.setup-volumes.sh
docker-dev/langs/Python/bin/20.build.sh
docker-dev/langs/Python/bin/30.run.sh

La soluzione della CR è **applicabile**, ma non è ancora sufficientemente corretta e definita per passare direttamente all'implementazione. L'architettura — catalogo `services/`, resolver Perl generico, template Template::Toolkit e Compose come backend — è coerente con il comando attuale. Sono però necessarie correzioni su ordine della pipeline, naming Compose, ciclo di vita del container di sviluppo, generazione YAML e compatibilità con gli argomenti Docker esistenti. [1] [2] [3]

Ho esaminato la CR e i nove file applicativi allegati. Ho anche verificato la sintassi dei sette script shell con `bash -n`: tutti passano. Non ho eseguito build delle immagini o avviato container: la verifica è statica e documentale, non un collaudo runtime.

1. Esito sintetico

Area	Esito
Catalogo `services/<tipo>/`	Impostazione valida
Perl + `YAML::XS` + Template::Toolkit	Scelta coerente
Separazione resolver / logica di servizio	Valida, ma da formalizzare meglio
Compatibilità della pipeline	Da correggere: `40.services.sh` arriva dopo `30.run.sh`
Nome progetto Compose	Errato negli esempi: `dev.python.foo` non è ammesso
Container `dev` effimero	Contraddizione tra `up -d` generale e `run --rm dev`

Area	Esito
Generazione dei template	Non supportata dall'attuale `apply-templates.sh`
Persistenza `venv` e `uv-python`	Riproducibile, purché entrambi siano volumi esterni
Compatibilità della porta 8000	Non garantita da `compose run` senza `--service-ports`
Manifest, escaping e aggregazione YAML	Contratto incompleto
Operazioni `--down`, `--config`, `--dry-run`	Richiedono flussi dedicati

Le criticità derivano dal confronto tra quanto prescritto dalla CR e ciò che eseguono effettivamente gli script. [2] [3] [4] [5] [1]

2. Problemi bloccanti

2.1 `40.services.sh` viene eseguito troppo tardi

Il runner attuale individua gli script numerati e li ordina con `sort -z -V`. Aggiungere semplicemente `40.services.sh` produrrebbe:

```
10.setup-volumes.sh
20.build.sh
30.run.sh
40.services.sh
```

Ma `30.run.sh` avvia il container in foreground e attende la sua terminazione. La configurazione dei servizi sarebbe generata solo dopo l'uscita dalla sessione di sviluppo, non prima come previsto dalla CR. [3] [1] [2]

Correzione consigliata:

```
10.setup-volumes.sh
20.build.sh
25.prepare-services.sh
30.run.sh
```

`25.prepare-services.sh` dovrebbe essere un wrapper sottile del resolver comune:

```
docker-dev/bin/docker-dev-services.pl
```

Se vuoi conservare il nome `40.services.sh`, occorre cambiare esplicitamente il runner affinché distingua preparazione ed esecuzione. Per la prima versione, rinumerare il wrapper è la modifica meno invasiva.

2.2 `dev.python.foo` non è un project name Compose valido

La CR propone:

```
docker compose -p dev.python.foo ...
```

Docker Compose ammette nel project name solo lettere minuscole, cifre, trattini e underscore, con una lettera minuscola o cifra iniziale. I punti non sono ammessi.[6] [1]

Vanno separati i nomi:

```
IMAGE_NAME           = dev.python
CONTAINER_NAME       = dev.python.foo
COMPOSE_PROJECT_NAME = dev-python-foo
VOLUME_NAME          = venv.dev.python.foo
```

Non occorre rinominare il volume esistente: Compose consente di specificarne il nome Docker effettivo con `volumes.<chiave>.name`.[7]

Inoltre, la CR deve definire una politica contro le collisioni: progetti diversi con lo stesso basename `foo`, o nomi diversi che producono la stessa normalizzazione, possono condividere involontariamente identità o volumi. L'attuale `P_NAME` deriva dal basename e normalizza alcuni caratteri.[2]

2.3 `up -d` generale e `run --rm dev` creano due container di sviluppo

La CR prescrive prima:

```
docker compose up -d
```

e poi una sessione:

```
docker compose run --rm dev
```

Il primo comando avvia anche il servizio `dev`; il secondo crea un container aggiuntivo one-off. Non è il ciclo di vita desiderato.[8] [9] [1]

Questo è particolarmente rilevante per il tuo entrypoint: senza argomenti apre direttamente Bash interattivo; con argomenti esegue `exec "$@"`. Non è un entrypoint progettato per un container residente con `sleep infinity`.[10]

Per mantenere il comportamento attuale, sceglierei:

```
avvio dei soli servizi selezionati in background
                  ↓
container dev one-off in foreground
                  ↓
rimozione del solo container dev all'uscita
```

```
                    ↓
servizi disponibili fino a --down
```

Ad esempio, a configurazione già validata:

```
docker compose "${compose_args[@]}" up -d db cache

docker compose "${compose_args[@]}" \
  run --rm --service-ports dev "${CONTAINER_CMD[@]}"
```

La politica di attesa degli healthcheck deve essere esplicita e verificata sulla versione minima di Compose supportata. I test di accettazione non devono più richiedere che un `up -d` generale avvii anche `dev`. [11] [1]

2.4 `apply-templates.sh` non renderizza Template::Toolkit

La CR afferma che `apply-templates.sh` genererà `.docker-dev/compose.yaml` dal template `compose.yaml.tt`.

Attualmente lo script:

- copia i file del template mantenendo il percorso relativo;
- evita la sovrascrittura salvo `-o`;
- compone `.docker_container/.bashrc` mediante concatenazione;
- non invoca Perl né Template::Toolkit. [4] [1]

Aggiungere `compose.yaml.tt` nei template Python lo farebbe quindi copiare, non renderizzare.

Consiglio di mantenere separate le responsabilità:

Componente	Responsabilità
`apply-templates.sh`	Consegna dei file del progetto e composizione `.bashrc`
Renderer Perl comune	Rendering del Compose base e dei template servizio
Resolver servizi	Selezione, validazione e risoluzione dei servizi
`30.run.sh`	Esecuzione legacy oppure Compose

La CR deve precisare quali template vengono copiati e quali vengono letti direttamente dal catalogo centrale, senza consegnarli al progetto.

3. Compatibilità con l'ambiente Python attuale

3.1 Il Compose base deve riprodurre `/app`, non i precedenti esempi `/workspace`

I file attuali concordano su:

```
working directory: /app
virtualenv:        /app/.venv
cache Python uv:   /home/<utente>/.local/share/uv/python
bashrc:           /app/.docker_container/.bashrc
```

Il Dockerfile imposta già `USER`, `WORKDIR`, `PATH` ed `ENTRYPOINT`. Il Compose base deve rispettare queste impostazioni, senza introdurre arbitrariamente `/home/dev`, un altro utente o un entrypoint alternativo. [12] [3] [10]

Uno scheletro coerente sarebbe:

```
services:
  dev:
    image: dev.python
    hostname: dev.python.foo
    stdin_open: true
    tty: true
    working_dir: /app
    volumes:
      - type: bind
        source: /percorso/assoluto/del/progetto
        target: /app
      - type: volume
        source: python-venv
        target: /app/.venv
      - type: volume
        source: uv-python
        target: /home/nomeutente/.local/share/uv/python
    ports:
      - "8000:8000"

volumes:
  python-venv:
    external: true
    name: venv.dev.python.foo

  uv-python:
    external: true
    name: uv-python
```

I valori sarebbero naturalmente generati dal contesto risolto, non hardcoded.

Non aggiungerei `command: ["sleep", "infinity"]`: senza comando, l'entrypoint attuale deve continuare a scegliere la shell con la `.bashrc` di progetto. [10]

3.2 La porta 8000 deve essere preservata

Il template `docker-run-extra-args` contiene:

```
--rm -it -p 8000:8000
```

Quindi la compatibilità non riguarda solo `--rm -it`, ma anche la pubblicazione della porta 8000.[5]

`docker compose run` non pubblica automaticamente le porte dichiarate nel servizio: serve `--service-ports`, oppure una pubblicazione esplicita con `-p`.[9]

La CR deve aggiungere questa condizione ai test di regressione. Un eventuale passaggio a `127.0.0.1:8000:8000` sarebbe una modifica intenzionale dell'esposizione di rete, non una riproduzione identica della configurazione corrente.

3.3 I volumi esterni devono essere creati prima dell'avvio

Per `venv` e `uv-python`, dichiarare `external: true` è coerente con `10.setup-volumes.sh`. Compose non crea un volume esterno e fallisce se questo manca; non lo elimina neppure con `down -v`.[13] [14] [7]

La CR, però, mostra anche un volume PostgreSQL esterno senza includere chiaramente l'esecuzione del relativo setup nella sequenza operativa. Questo passaggio deve essere obbligatorio quando il manifest richiede volumi esterni.[1]

Non va applicata automaticamente ai database la stessa ownership dell'utente host usata per il virtualenv: l'inizializzazione appartiene al servizio, tramite il suo hook.

4. Il resolver deve restare generico anche negli alias e nei manifest

4.1 Alias dichiarativi, non hardcoded in Perl

Per rispettare davvero il requisito "nessuna logica specifica di servizio", anche le associazioni:

```
db → postgres
cache → redis
```

dovrebbero essere dati di configurazione, non un `case` Bash o un hash statico nello script Perl.

Una soluzione è:

```
# configurazione del progetto o catalogo alias
aliases:
  db: postgres
  cache: redis
```

Il resolver conosce solo il meccanismo di risoluzione; il progetto sceglie quale implementazione corrisponde a `db`. La CR attuale definisce la tabella, ma non il luogo in cui viene mantenuta né come si sovrascrive. [1]

4.2 La grammatica CLI è ambigua

Gli esempi mescolano:

```
--with=db,cache
```

e:

```
--with=db:tag=16,host_port=15432
```

La virgola separa sia i servizi sia i parametri. La CR deve fissare una grammatica non ambigua prima di implementare il parser. [1]

Una proposta semplice:

```
docker-dev --target=Python/foo \
  --with=db \
  --with=cache \
  --service-option=db.tag=16 \
  --service-option=db.host_port=15432
```

Puoi mantenere `--with=db,cache` come abbreviazione per soli nomi, mentre i parametri usano opzioni separate.

Il parsing deve inoltre fermarsi a `--`, preservando il comando e ogni suo argomento senza reinterpretarli.

4.3 Il manifest è troppo restrittivo in alcuni punti e incompleto in altri

Rendere obbligatori `container_port` e `connection_env` esclude servizi che non espongono porte o non richiedono variabili di connessione. Al contrario, `build`, `volumes`, `networks` e `depends_on` sono indicati come opzionali senza uno schema definito. [1]

Occorre formalizzare almeno:

- versione dello schema;
- riferimento immagine completo oppure repository e tag;
- modalità immagine: locale costruita / upstream;
- parametri ammessi e relativi tipi;
- profili come lista;
- zero o più volumi;
- hook di preparazione e build;

- integrazione con `dev`;
- politica delle porte e della persistenza.

Il manifest Redis contiene `REDIS_ARGS`, ma i file allegati non definiscono un'immagine o entrypoint Redis che consumi quella variabile. L'esempio è quindi una proposta, non un comportamento verificato. [1]

5. Generazione YAML: va corretta la strategia di aggregazione

5.1 Non concatenare documenti con `services:` ripetuto

Ogni template servizio nella CR produce:

```
services:
  ...
```

Concatenare due risultati genererebbe due chiavi top-level `services`. Non è un'aggregazione affidabile. [1]

Il flusso corretto è:

```
manifest YAML
    ↓ parsing e validazione
contesto risolto
    ↓ rendering Template::Toolkit
frammento YAML
    ↓ parsing del frammento
struttura Perl
    ↓ aggregazione controllata
un solo documento Compose
    ↓ serializzazione YAML
compose.services.yaml
```

Il risultato deve contenere una sola mapping `services`, una sola `volumes` ed eventualmente una sola `networks`.

Servono regole esplicite per i conflitti: due istanze con lo stesso nome, due servizi che assegnano la stessa variabile di connessione, o due dichiarazioni incompatibili dello stesso volume devono generare errore, non una sovrascrittura silenziosa.

5.2 I valori non possono essere interpolati senza escaping

Questo template:

```
[% pair.key %]: [% pair.value %]
```

non garantisce YAML valido o conservazione del tipo per valori con `:`, `#`, newline, virgolette o stringhe interpretabili come booleani. Anche `healthcheck_test`, essendo una lista, non va

passato direttamente come arrayref a un'interpolazione testuale. [1]

La soluzione deve prevedere filtri di serializzazione YAML/JSON per scalari e strutture, oppure costruzione strutturata delle parti dinamiche.

Inoltre, occorre distinguere il rendering Template::Toolkit dalla successiva interpolazione Compose delle espressioni `${...}`.

5.3 `STRICT => 1` è corretto, ma il contesto deve essere completo

Template::Toolkit supporta effettivamente `STRICT => 1`: una variabile non definita causa un'eccezione. [15]

Nel template della CR viene usato:

```
[% IF healthcheck %]
```

ma `healthcheck` non compare nell'elenco delle variabili promesse al template. Con modalità strict è una discrepanza da correggere. [1]

Preferirei flag espliciti e sempre presenti:

```
has_healthcheck = 0 oppure 1
has_data_volume = 0 oppure 1
publish_port   = 0 oppure 1
```

6. Integrazione Bash: aspetti mancanti

6.1 I figli non ricevono automaticamente le variabili locali

Il runner esegue gli script come processi separati passando:

```
PROJECT_DIR IMAGE_NAME CONTAINER_NAME [comando...]
```

Variabili assegnate ma non esportate in `docker-dev.sh` non costituiscono un contratto di comunicazione con questi processi. Questo riguarda sia le nuove opzioni Compose sia gli override di `USER_`, `UID_`, `GID_` e altri valori risolti. [16] [13] [2]

Per facilitare la futura migrazione a Perl, suggerisco un contesto JSON esplicito:

```
project_dir
project_type
project_environment
project_name
image_name
container_name
compose_project_name
host_user
```

```
host_uid
host_gid
volume_names
requested_services
```

Il comando da eseguire può continuare a essere passato come array di argomenti, senza conversione in una stringa shell.

6.2 `DOCKER_RUN_EXTRA_ARGS` non può essere ignorato silenziosamente

Il percorso legacy accetta opzioni arbitrarie tramite ambiente o file, con esclusione convenzionale di nome e immagine. La CR non definisce cosa succede a queste opzioni in modalità Compose. [17] [1]

Per una prima implementazione:

- mantenere invariato il percorso legacy;
- trasferire esplicitamente i default conosciuti nel Compose base;
- usare `compose.user.yaml` per le personalizzazioni Compose;
- segnalare chiaramente opzioni legacy non supportate, anziché ignorarle;
- evitare un traduttore automatico generale `docker run → Compose`.

6.3 `--down`, `--config` e `--dry-run` non devono eseguire tutta la pipeline

Se questi comandi attraversassero il flusso normale, potrebbero creare volumi, costruire immagini o persino avviare il container prima dell'operazione richiesta. Il main attuale non distingue operazioni amministrative dalla sessione di sviluppo. [2] [1]

Serve una separazione anticipata:

Operazione	Comportamento
Sessione	Prepara, valida, crea risorse, avvia servizi, esegue `dev`
`--config`	Risolve/renderizza e mostra il modello, senza build o avvio
`--dry-run`	Mostra piano e configurazione, senza modificare risorse Docker
`--down`	Usa la configurazione dello stack precedente e lo arresta
`--build-service`	Opera sul catalogo servizi, anche senza target

Per `--down`, non bisogna rigenerare un file vuoto solo perché l'utente non ha ripetuto `--with`. Va conservata la configurazione applicata allo stack, non soltanto un hash dei manifest.

7. Altri rilievi sul codice attuale

Questi punti precedono Compose e dovrebbero essere registrati come problemi della baseline, non attribuiti alla nuova integrazione.

Risoluzione delle opzioni prima dell'applicazione dei template

`docker-dev.sh` risolve gli extra args prima di applicare i template. `30.run.sh` li risolve nuovamente dopo la copia, ma l'esito dipende anche dall'eventuale variabile già esportata nell'ambiente. Conviene distinguere un override esplicito dell'utente da un fallback calcolato internamente.[3] [17] [2]

Ownership verificata solo alla creazione del volume

`10.setup-volumes.sh` ritorna immediatamente se il volume esiste. Non verifica né ripara ownership e permessi dei volumi già presenti, e non rileva un'inizializzazione precedente fallita. La CR dovrebbe descrivere precisamente questo comportamento e trattare la riparazione come operazione esplicita.[13]

Build arg `TZ` non utilizzato dal Dockerfile

`20.build.sh` passa `--build-arg TZ=...`, ma il Dockerfile non dichiara `ARG TZ` e imposta `ENV TZ=Europe/Rome`. Un override di `TZ` nello script non cambia quindi la timezone dell'immagine come ci si potrebbe aspettare.[12] [16]

Sincronizzazione del virtualenv

L'entrypoint salta `uv sync` quando il virtualenv non è vuoto, salvo `UV_SYNC_ALWAYS=1`. Compose deve preservare tale comportamento o cambiarlo con una decisione separata e documentata; non basta ricreare il container per aggiornare le dipendenze persistenti.[10]

8. Flusso corretto proposto

Per la sessione di sviluppo:

```
1. Parsing completo della CLI, rispettando --
2. Risoluzione del progetto e delle identità
3. Scelta backend legacy / Compose
4. Applicazione dei file template del progetto
5. Risoluzione e rendering Compose, senza effetti Docker
6. Validazione della configurazione completa
7. Conferma del piano
8. Setup volumi di linguaggio
9. Build immagine di sviluppo se assente
10. Setup/build/pull dei servizi tramite hook dichiarati
11. Avvio dei soli servizi selezionati e attesa readiness
12. docker compose run --rm --service-ports dev [comando...]
13. Conservazione dello stato applicato per --down
```

Per i servizi, il resolver continua a essere generico: esegue hook dichiarati e interpreta uno schema comune; non contiene comandi PostgreSQL o Redis.

Il file opzionale `compose.user.yaml` deve essere aggiunto all'array `-f` solo se esiste. L'esempio della CR lo passa sempre, pur dichiarandolo facoltativo. [1]

9. Completezza dei materiali e conclusione

I materiali allegati sono sufficienti per questa revisione e per definire le modifiche principali. Non sono però sufficienti per dichiarare verificata l'intera esecuzione: mancano `.settings/.bashrc` e `.settings/.gitignore`, richiesti da `apply-templates.sh`, nonché le future definizioni reali dei servizi e la versione di Compose da supportare. [4] [1]

La CR può essere approvata come proposta architetturale, ma deve essere revisionata prima dell'implementazione almeno su questi punti:

1. preparazione servizi prima di `30.run.sh`;
2. project name Compose separato dai nomi Docker esistenti;
3. avvio in background dei soli servizi, non del container `dev`;
4. renderer Perl distinto dalla semplice copia dei template;
5. preservazione di `/app`, entrypoint, `.bashrc`, volumi e porta 8000;
6. grammatica CLI non ambigua e contesto esplicito tra processi;
7. aggregazione strutturata e serializzazione sicura del YAML;
8. flussi amministrativi e stato persistito per `--down`.

Alcune imprecisioni provengono dai miei esempi precedenti, soprattutto il project name con punti e la sequenza `up -d` generale seguita da `run --rm dev`: vanno corrette nella CR, non replicate nel codice.

⁂

1. change-request-docker-compose-docker-dev.md
2. docker-dev.sh
3. 30.run.sh
4. apply-templates.sh
5. docker-run-extra-args
6. https://docs.docker.com/compose/how-tos/project-name/
7. https://docs.docker.com/reference/compose-file/volumes/
8. https://docs.docker.com/compose/intro/compose-application-model/
9. https://docs.docker.com/reference/cli/docker/compose/run/
10. entrypoint.Python.sh
11. https://docs.docker.com/reference/cli/docker/compose/up/
12. Dockerfile

13. 10.setup-volumes.sh
14. https://docs.docker.com/reference/cli/docker/compose/down/
15. https://metacpan.org/dist/Template-Toolkit/view/lib/Template/Manual/Config.pod
16. 20.build.sh
17. docker-run-extra-args.sh
18. https://man.archlinux.org/man/extra/perl-yaml-libyaml/YAML::XS.3pm.en
19. https://www.netdata.cloud/academy/docker-compose-networking-mysteries/
20. https://perl.mines-albi.fr/perl5.8.5/site_perl/5.8.5/sun4-solaris/Template/FAQ.html
21. https://stackoverflow.com/questions/571898/can-perls-template-toolkit-warn-on-undefined-values
22. https://dev.to/meghasharmaaaa/docker-compose-commands-p55
23. https://devtodevops.com/blog/docker-compose-network-name-without-prefix/
24. https://github.com/docker/compose/issues/4002
25. https://github.com/cpan-authors/Template2/issues/75
26. https://appletree.or.kr/quick_reference_cards/Perl/Template Toolkit Quick Reference Card-fullpage.pdf
27. https://template.mailkit.com/modules/Template.php
28. https://context7.com/docker/compose
29. https://docs.docker.com/reference/compose-file/networks/
30. https://docs.docker.com/reference/cli/docker/compose/build/
31. https://docs.docker.com/compose/how-tos/multiple-compose-files/
32. https://docs.docker.com/compose/gettingstarted/
33. https://docs.docker.com/reference/cli/docker/compose/
34. https://docs.docker.com/reference/compose-file/services/
35. https://docs.docker.com/compose/how-tos/multiple-compose-files/merge/
36. https://docs.docker.com/engine/storage/volumes/
37. https://docs.docker.com/reference/compose-file/
38. https://docs.docker.com/compose/how-tos/multiple-compose-files/extends/